

# Day 15 — Exception Subprocess

## 1. What Is an Exception Subprocess?

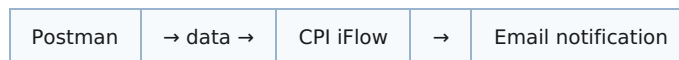
If we want to handle any errors in CPI, we use the Exception Subprocess.

### Why Do We Need Error Handling?

When you develop any resource (iFlow) and it becomes **operational** (live), errors will eventually happen. The question is: **what will you do in SAP CPI when an error occurs?**

CPI is a middleware tool — data moves from one system to a 2nd system. If an error happens and **the source system's people don't know about it, that is a real problem** (they think the data arrived, but it didn't). So sometimes we need to **send the error details to the source system people**, or at least we need to know **what happened**.

### Scenario from Class



If it somehow fails in between, what do we do? **You must know the status — whether it failed or succeeded.** To handle this situation, we use the **Exception Subprocess**.

## 2. Real-Time Requirement — Email Alert on Error

**Scenario:** if you get any error, you need to send an **email alert to the Support Team** (or whoever is responsible).

### Fields the Email Must Contain (at least)

| # | Field                                        | How to Get It                                                                    |
|---|----------------------------------------------|----------------------------------------------------------------------------------|
| ① | <b>iFlow name</b> (which one failed)         | <code>\${camelId}</code>                                                         |
| ② | <b>At what time</b>                          | <code>\${date:now:yyyy-MM-dd'T'HH:mm:ss}</code> (or the IST version from Day 13) |
| ③ | <b>MsgID</b> (to find it in Monitor)         | <code>\${property.SAP_MessageProcessingLogID}</code>                             |
| ④ | <b>What is the exception</b> (error content) | <code>\${exception.message}</code>                                               |

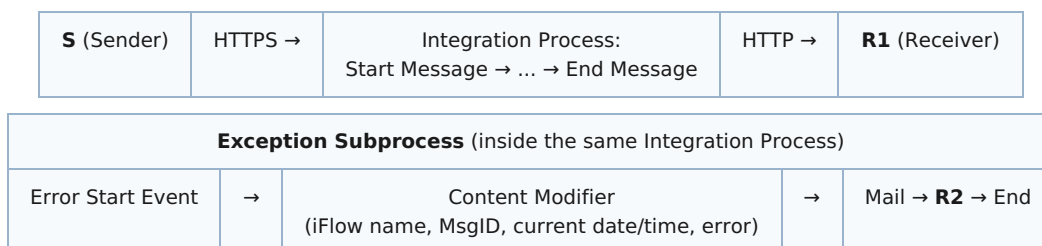
**Small correction:** your notes abbreviate the MsgID property as `property.SAP_MPLID`. The real property name is **SAP\_MessageProcessingLogID** (the same one used on Day 13) — use `${property.SAP_MessageProcessingLogID}`, otherwise the value will come out empty.

### Error Content vs. Detail Log

| Purpose                                  | Expression                            |
|------------------------------------------|---------------------------------------|
| <b>Error content</b> (short, readable)   | <code>\${exception.message}</code>    |
| <b>Detail log</b> (full technical trace) | <code>\${exception.stacktrace}</code> |

(Both are Camel Simple expressions — case-sensitive, so write them exactly in lowercase as shown.)

## 3. The iFlow Design (Diagram)



**In the sketch:** R1's HTTP call is deliberately given a **wrong configuration** so it fails — that failure triggers the Exception Subprocess, which builds the alert text in a Content Modifier and sends it through the Mail adapter to R2 (the support mailbox).

**Tip for practice:** this is the easiest way to test your design — deliberately break the receiver (wrong URL or credentials), send a message from Postman, and watch the alert email arrive. Reuse the Gmail/Mail adapter setup from Day 8.

#### 4. Not Only Email — Ticketing Tools Too

If we want to send an email alert, we use the Exception Subprocess — but **not only email alerts**. If we get an error, we can also create an **Incident / Ticket** in a tool like **ServiceNow** (or similar ticketing tools): **error → Exception Subprocess → create ticket → ServiceNow**.

##### Why Create the Ticket Inside the Exception Subprocess?

**Real-time scenario question:** if you want to send a ticket to ServiceNow on errors, why do it in the Exception Subprocess?

**Answer:** if you did it as a **normal call** in the main flow, **every successful message would also create a ticket** — which is **not encouraged X**. The requirement is **only errors → ticket**, so the call must sit in the **Exception Subprocess**, which runs only when something fails.

#### 5. Key Rules to Remember

- **In an Exception Subprocess you can use almost any Palette option (function)** — Content Modifier, Script, Process Call, Request-Reply, Mail/HTTP receivers, etc.
- **You cannot call an Exception Subprocess explicitly** — it is called **internally, only when there is an error**.
- **Where can it exist?** Only inside the **Main Integration Process** or a **Local Integration Process** (as in Days 9 and 14). It triggers when an error occurs in that process — we cannot call it directly.

**Reminder from Day 14:** each Integration Process / Local Integration Process needs its **own** Exception Subprocess — one does not cover the others. A good design is a single reusable **"send error alert" Local Integration Process**, called from every Exception Subprocess.

#### 6. Important: How the Exception Subprocess Should End

Your notes don't mention this, but it is the most common real-project decision with Exception Subprocesses, so it is worth adding (verified against SAP's documentation):

| Ends With               | Message Status in Monitor                                                  | What the Sender Gets                                                               |
|-------------------------|----------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| End Event / End Message | <b>Completed</b> — the error is "handled", so the message looks successful | You can set a <b>custom response</b> and HTTP status code (e.g., 400)              |
| Error End Event         | <b>Failed</b>                                                              | Standard <b>HTTP 500</b> error with the MPL ID — a custom error text cannot be set |
| Escalation End Event    | <b>Escalated</b> (a separate status you can filter on in Monitor)          | Standard HTTP 500 error, like Error End                                            |

**Practical rule:** for an **alert design**, the email/ticket is sent *before* the end event, so it works with any of them. But if you end with a plain End Event, **a failed message shows as "Completed" in Monitor** — support may never notice it. Use **Error End Event** (or Escalation End) when you want the failure to stay visible; use **End Message** only when you deliberately want to send a clean custom error response back to the sender.

#### 7. Line in Your Notes I Could Not Interpret

Your last note reads: *"Only pass-through scenarios only → we will [use] multiple Main Integration Process in a single iFlow."*

**Possible meaning:** multiple Main Integration Processes in one iFlow (Day 14) would be used only for simple **pass-through** scenarios — each process just receives from its own sender and forwards, without much transformation. I'm not certain this is what your trainer meant, so I'd rather flag it than guess — worth asking in the next class.